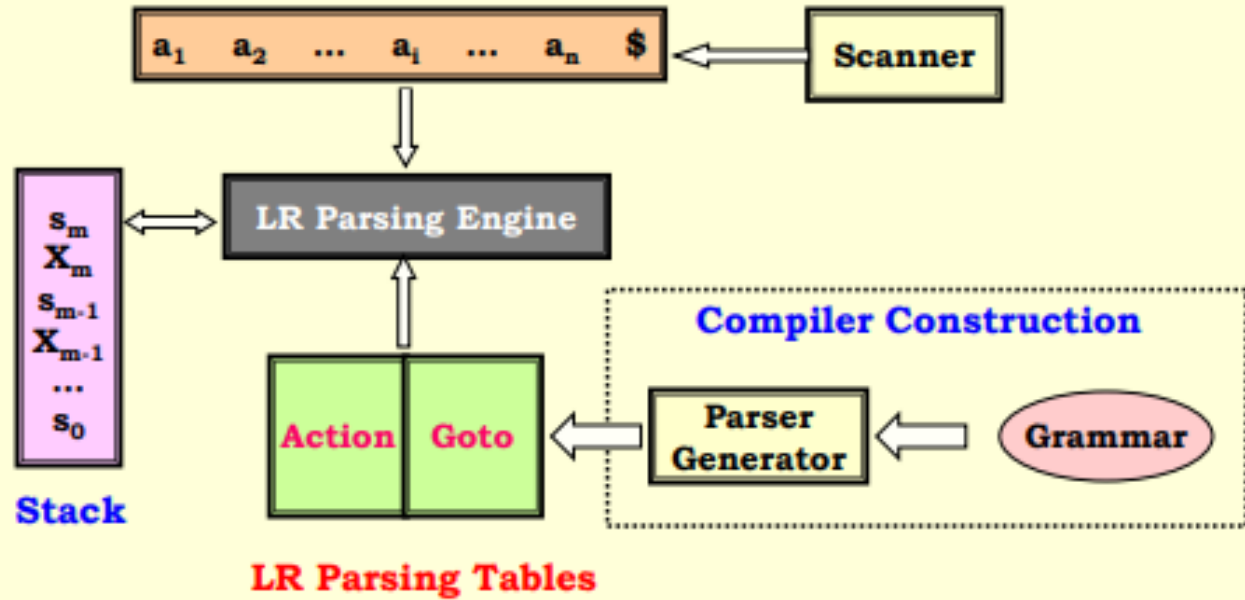


LR, SLR, CLR, LALR

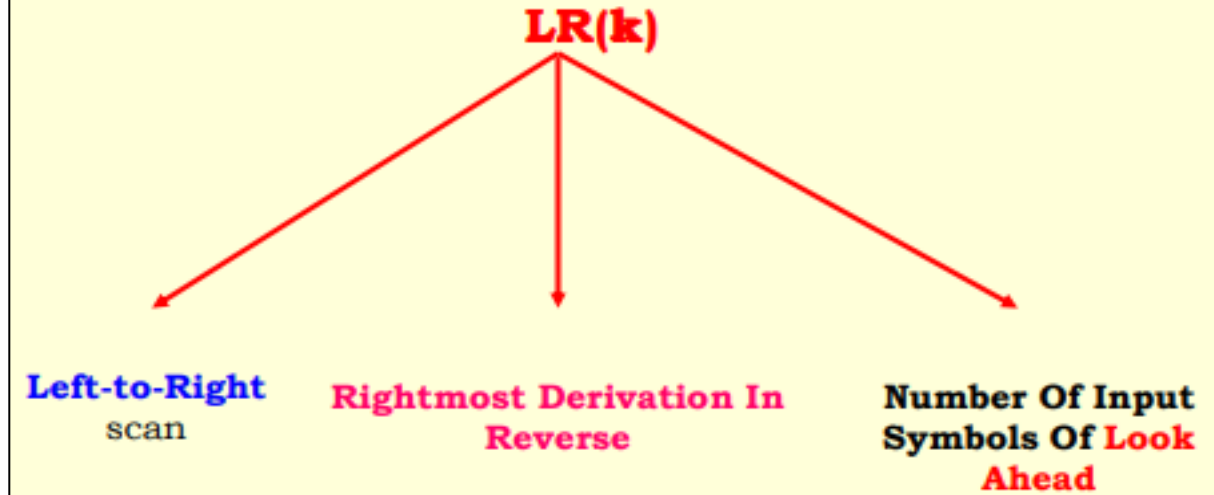
LR PARSING

- LR parsers are used to parse the **large class of context free grammars**.
- LR parsers are constructed to recognize all Programming Languages
- The LR-parsing is Non-Backtracking ShiftReduce Parser
- An LR parser can detect a syntactic errors ▪ It scans input string from left-to-right and use left most derivation in reverse
- This technique is called **LR(k) parsing**.
- **L** is **left-to-right scanning** of the input.
- **R** is for constructing a **right most derivation in reverse**.
- **k** is the **number of input symbols of lookahead** that are used in making parsing decisions.
- The letter k is usually one and is frequently omitted.

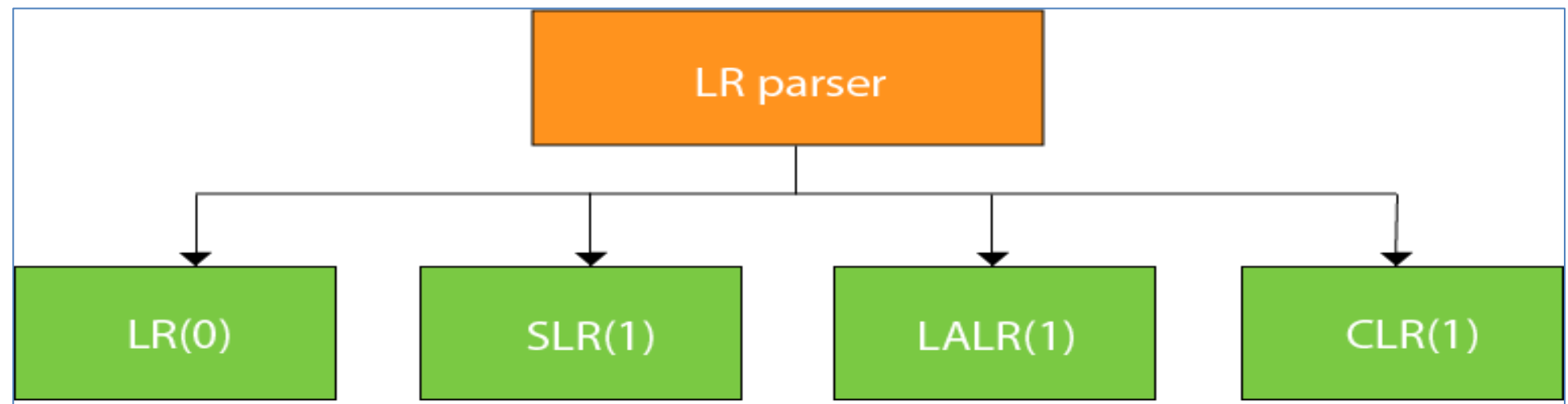
Input



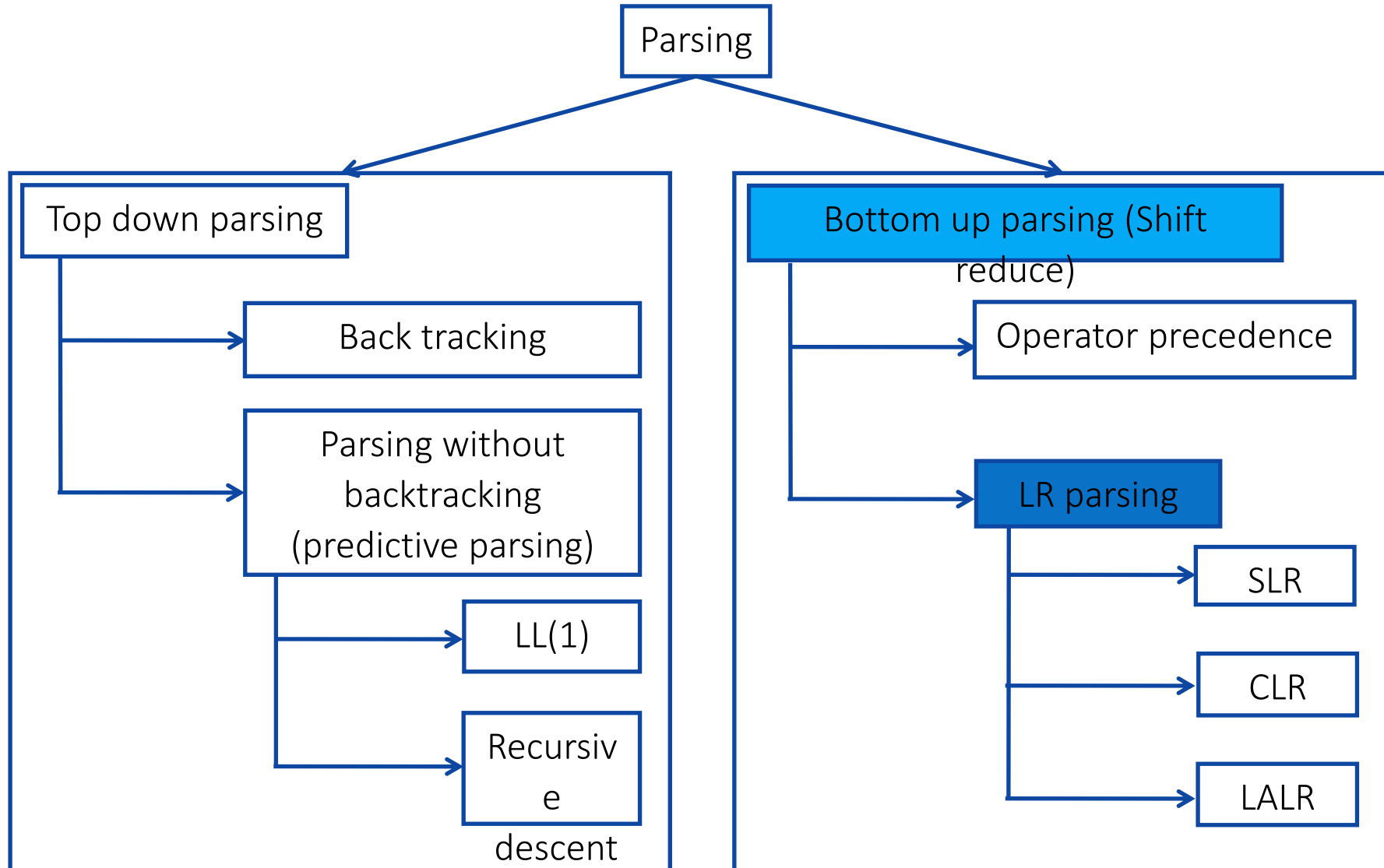
LR Parsers



- LR parsing is divided into four parts



Parsing Methods



Computation of closure & goto function

Closure(I):

- If 'I' is a *set of items* for a grammar, G then closure of I (closure (I)) is set of items constructed from I by applying following 3 rules:
 1. **Construct augmented grammar.**
 2. **Initially, every item in I is added to closure(I).**
 3. **If $A \rightarrow \alpha \cdot B \beta$ is in closure (I) and $B \rightarrow \gamma$ is a production, then add the item $B \rightarrow \cdot \gamma$ to I, if it is not already in existence, we apply this rule until no more new items can be added to closure(I).**

Example:

$E \rightarrow E+T \mid T$

$T \rightarrow T * F \mid F$

$F \rightarrow (E) \mid id$

CLOSURE

$E' \rightarrow .E$

$E \rightarrow .E+T$

$E \rightarrow .T$

$T \rightarrow .T * F$

$T \rightarrow .F$

$F \rightarrow .(E)$

$F \rightarrow .id$

Computation of closure & goto function

Goto(I,X)

- The function goto can be defined as,
 - If there is a production $A \rightarrow \alpha \bullet B \beta$ then,
$$\text{goto}(A \rightarrow \alpha \bullet B \beta, B) = A \rightarrow \alpha B \bullet \beta.$$
 - Simply shifting of “•” one position ahead over grammar symbol.

Example:

$E \rightarrow E+T \mid T$

$T \rightarrow T * F \mid F$

$F \rightarrow \text{id} \mid (E)$

CLOSURE

$E' \rightarrow .E$

$E \rightarrow .E+T$

$E \rightarrow .T$

$T \rightarrow .T * F$

$T \rightarrow .F$

$F \rightarrow .(E)$

$F \rightarrow .\text{id}$

CLOSURE

$E' \rightarrow .E$

$E \rightarrow .E+T$

$E \rightarrow .T$

$T \rightarrow .T * F$

$T \rightarrow .F$

$F \rightarrow .(E)$

$F \rightarrow .id$

goto(I₀, E)

I_1 $E' \rightarrow E.$
 $E \rightarrow E.+T$

goto(I₀, T)

I_2 $E \rightarrow T.$
 $T \rightarrow T.*F$

goto(I₀, F)

I_3 $T \rightarrow F.$

goto(I₀, (

I_4 $F \rightarrow (.E)$
 $E \rightarrow .E+T$
 $E \rightarrow .T$
 $T \rightarrow .T * F$
 $T \rightarrow .F$
 $F \rightarrow .(E)$
 $F \rightarrow .id$

goto(I₀, id)

I_5 $F \rightarrow id.$

goto(I₁, +)

I_6 $E \rightarrow E+.T$
 $T \rightarrow .T * F$
 $T \rightarrow .F$
 $F \rightarrow .(E)$
 $F \rightarrow .id$

goto(I₂, *)

I_7 $T \rightarrow T*.F$
 $F \rightarrow .(E)$
 $F \rightarrow .id$

goto(I₄, E)

I_8 $F \rightarrow (E.)$
 $E \rightarrow E.+T$

goto(I₄, T) ➔ I_2

goto(I₄, F) ➔ I_3

goto(I₄, (➔ I_4

goto(I₄, id) ➔ I_5

goto(I₆, T)

I_9 $E \rightarrow E+T.$
 $T \rightarrow T.*F$

goto(I₆, F) ➔ I_3

goto(I₆, (➔ I_4

goto(I₆, id) ➔ I_5

goto(I₇, F)

I_{10} $T \rightarrow T * F.$

goto(I₇, (➔ I_4

goto(I₇, id) ➔ I_5

goto(I₈,)

I_{11} $F \rightarrow (E).$

goto(I₈, +) ➔ I_6

goto(I₉, *) ➔ I_7

Steps to construct Simple LR (SLR) parser

1. Construct Canonical set of LR(0) items
2. Construct SLR parsing table
3. Parse the input string

Example: SLR(1)- simple LR

$S \rightarrow AA$

$A \rightarrow aA \mid b$

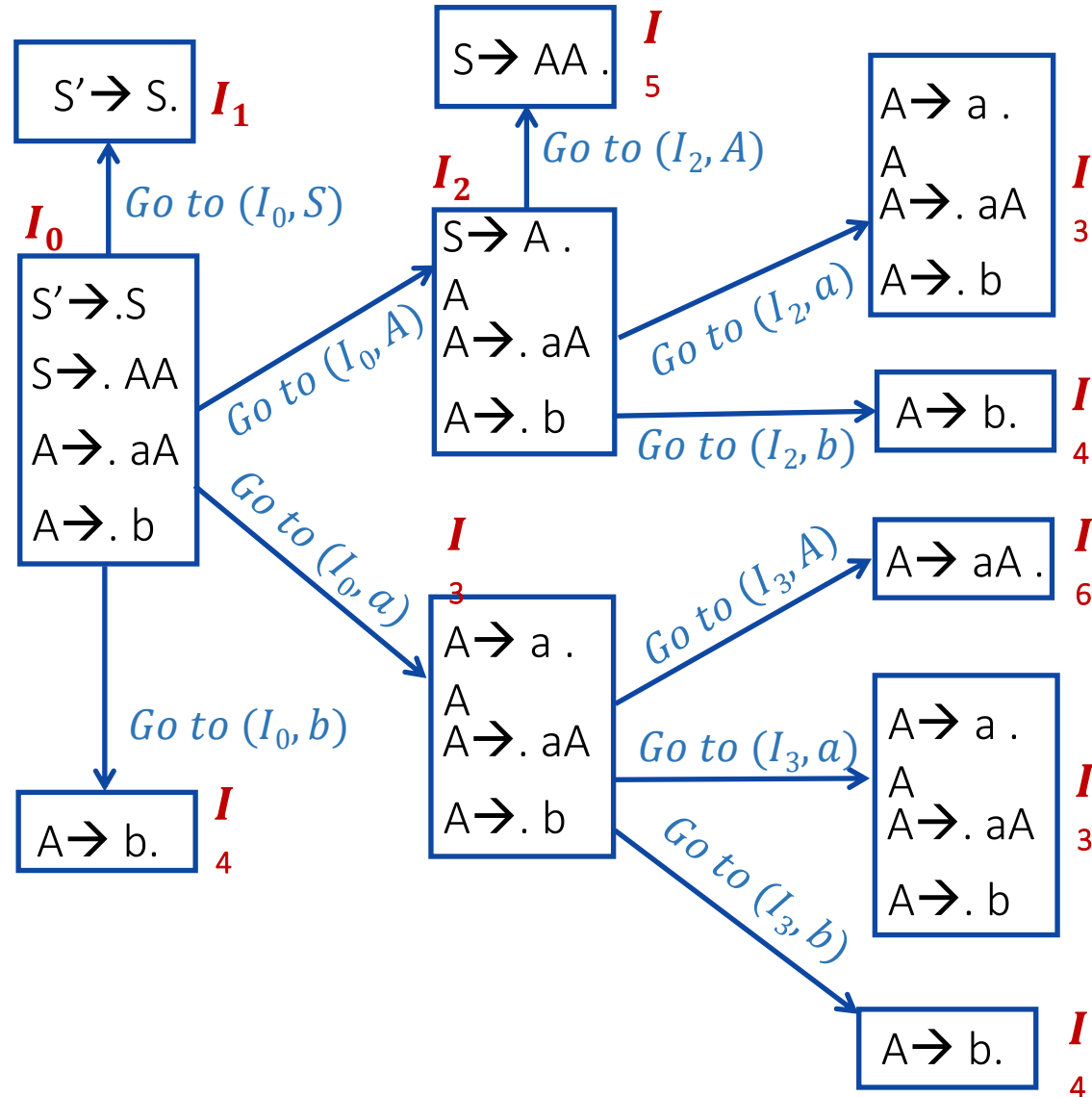
$S' \rightarrow S$

$S \rightarrow AA$

$A \rightarrow aA$

$A \rightarrow b$

Augmented
Grammar



LR(0) item set

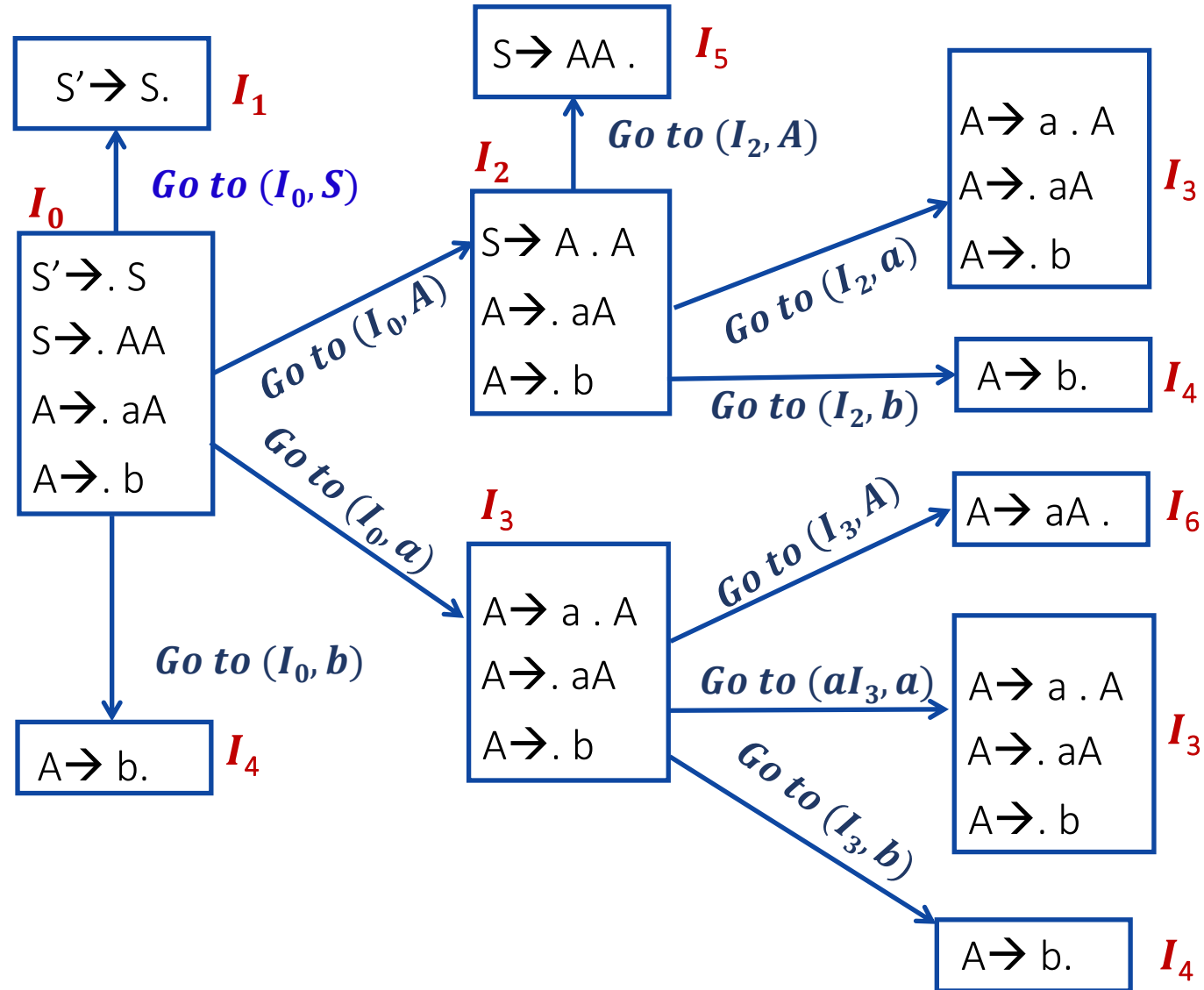
Rules to construct SLR parsing table

1. Construct $C = \{I_0, I_1, \dots, I_n\}$, the collection of sets of LR(0) items for G .
2. State i is constructed from I_i . The parsing actions for state i are determined as follow :
 - a) If $[A \rightarrow \alpha.a\beta]$ is in I_i and $GOTO(I_i, a) = I_j$, then set $ACTION[I_i, a]$ to "shift j". Here, 'a' must be a terminal.
 - b) If $[A \rightarrow \alpha.]$ is in I_i , then set $ACTION[I_i, a]$ to "reduce $A \rightarrow \alpha$ " for all 'a' in $FOLLOW(A)$; here, A may not be S' .
 - c) If $[S \rightarrow S.]$ is in I_i , then set action $[I_i, \$]$ to "accept".
3. The goto transitions for state, i are constructed for all non-terminals A using the *if $GOTO(I_i, A) = I_j$ then $GOTO[i, A] = j$.*
4. All entries not defined by rules 2 and 3 are made error.

Example: SLR(1)- simple LR

$Follow(S) = \{\$ \}$

$Follow(A) = \{a, b, \$ \}$



Item set	Action			Go to	
	a	b	\$	S	A
0	S3	S4		1	2
1			Accept		
2	S3	S4			5
3	S3	S4			6
4	R3	R3	R3		
5			R1		
6	R2	R2	R2		

$S \rightarrow S$ -----(0)

$S \rightarrow AA$ -----(1)

$A \rightarrow aA$ -----(2)

$A \rightarrow b$ -----(3)

Parse the string **aabb** using SLR parser

Stack	Input	Action	
0	aabb\$	Shift I0->a, S3	
0a3	abb\$	Shift I3 ->a, S3	
0a3a3	bb\$	I3->b, S4	
0a3a3 b 4	b\$	I4->b, R3	Pop 4, goto(I3,A)=6
0a3 a 3 A 6	b\$	I6->b, R2	
0 a 3 A 6	b\$	I6->b, R2	
0 A 2	b\$	I2->b, s4	
0A2 b 4	\$	I4->\$, r3	Pop 4, goto(I2,A)=5
0 A 2 A 5	\$	I5->\$,r1	Pop 5, goto(I0,S)=1
0 S 1	\$	Accept	

Parsing Table

	Action			Go to	
Item set	a	b	\$	S	A
0	S3	S4		1	2
1			Accept		
2	S3	S4			5
3	S3	S4			6
4	R3	R3	R3		
5			R1		
6	R2	R2	R2		

S-->S' -----(0)

S-->AA -----(1)

A-->aA -----(2)

A-->b -----(3)

Example:2

Construct SLR parsing table for the following grammar:

$S \rightarrow Aa \mid bAc \mid Bc \mid bBa$

$A \rightarrow d$

$B \rightarrow d$

Solution:

Step 1: Grammar Augmentation

$S' \rightarrow S$... rule 0

$S \rightarrow Aa$... rule 1

$S \rightarrow bAc$... rule 2

$S \rightarrow Bc$ rule 3

$S \rightarrow bBa$ rule 4

$A \rightarrow d$ rule 5

$B \rightarrow d$... rule 6

Step 2: Closure Operation

$S' \rightarrow .S$

$S \rightarrow .Aa$

$S \rightarrow .bAc$

$S \rightarrow .Bc$

$S \rightarrow .bBa$

$A \rightarrow .d$

$B \rightarrow .d$

Goto (i0, S) = i1

$S' \rightarrow S.$

Goto (i0,A) = i2

$S \rightarrow A.a$

Goto(i0,b) = i3

$S \rightarrow b.Ac$

$S \rightarrow b.Ba$

$A \rightarrow .d$

$B \rightarrow .d$

Goto(i0,B)=i4

$S \rightarrow B.c$

Goto(i0,d)=i5

$A \rightarrow d.$

$B \rightarrow d.$

Goto(i2,a)=i6

$S \rightarrow Aa.$

Goto(i3,A)=i7

$S \rightarrow bA.c$

Goto(i3,B)=i8

$S \rightarrow bB.a$

Goto(i3,d)=i5 //loop

Goto(i4,c)=i9

$S \rightarrow Bc.$

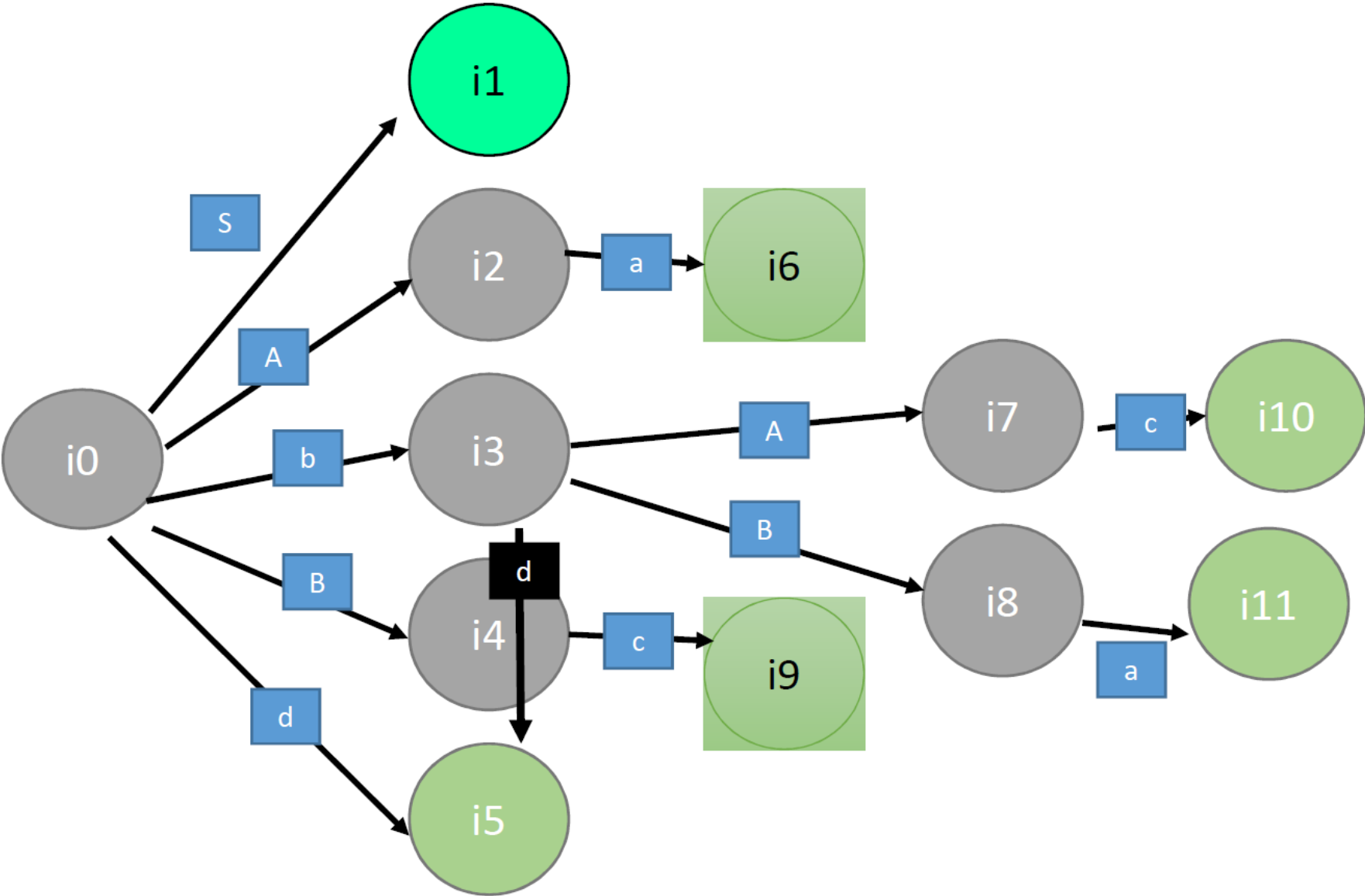
Goto(i7,c)=i10

$S \rightarrow bAc.$

Goto(i8,a)=i11

$S \rightarrow bBa.$

Cont... Canonical Form



Parsing table

$S \rightarrow Aa \mid bAc \mid Bc \mid bBa$
 $A \rightarrow d$
 $B \rightarrow d$
 $\text{Follow}(S) = \{\$ \}$
 $\text{Follow}(A) = \{a, c\}$
 $\text{Follow}(B) = \{a, c\}$

	a	b	c	d	\$	S	A	B
I0		S3		S5		1	2	4
I1					Accept			
I2	S6							
I3				S5			7	8
I4			S9					
I5	R5,R6		R5,R6					
I6					R1			
I7			S10					
I8	S11							
I9					R3			
I10					R2			
I11					R4			

Exercises

1. Construct SLR parsing table for the following grammar:

$E \rightarrow E + T \mid T$

$T \rightarrow T * F \mid F$

$F \rightarrow \text{id}$

2. Construct SLR parsing table for the following grammar and parse the input string $a * b + a$:

$E \rightarrow E + T \mid T$

$T \rightarrow TF \mid F$

$F \rightarrow F^* \mid a \mid b$

3. Construct SLR parsing table for the following grammar

$S \rightarrow L = R \mid R$

$L \rightarrow * R \mid \text{id}$

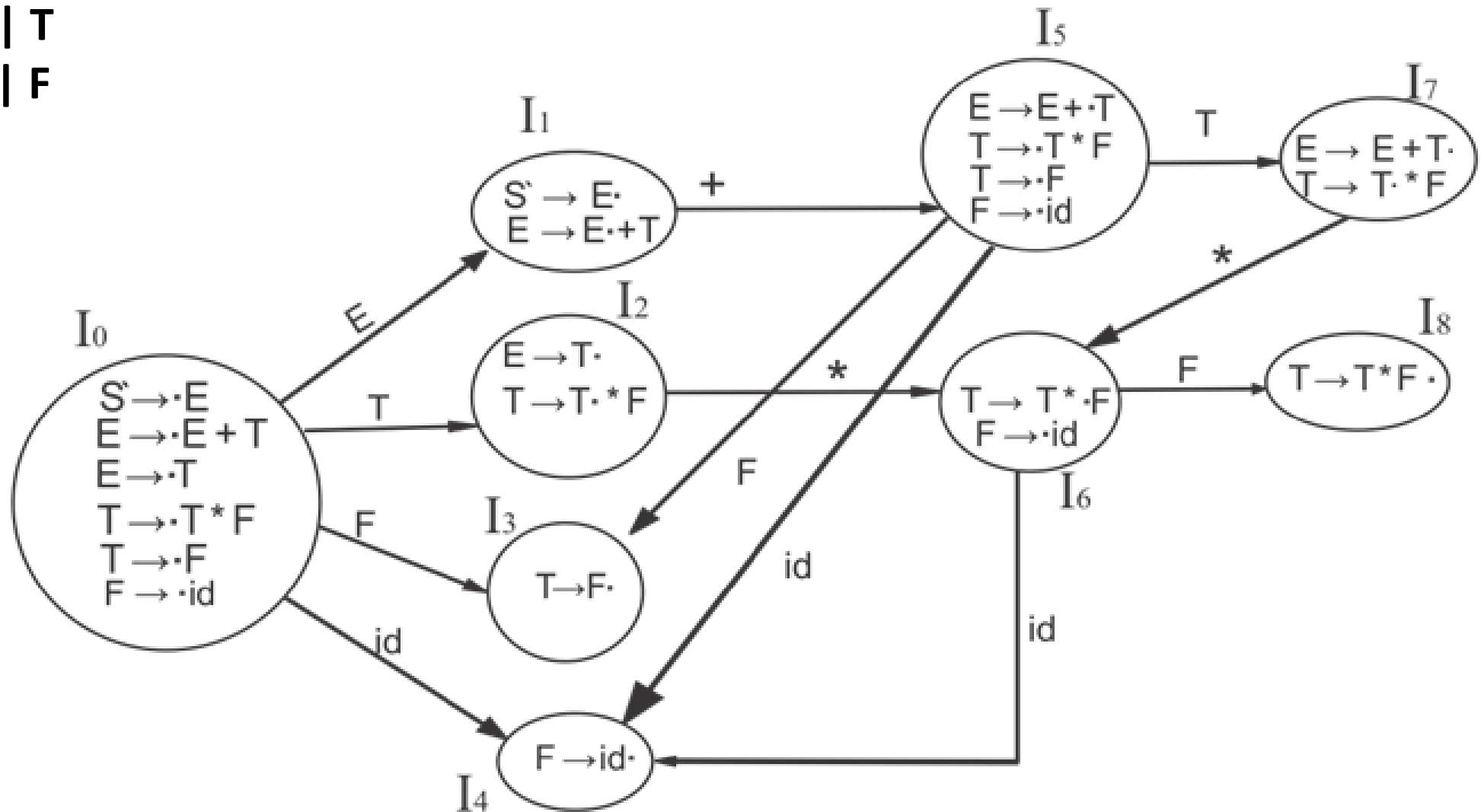
$R \rightarrow L$

Solution 1 – Canonical form

$E \rightarrow E + T \mid T$

$T \rightarrow T * F \mid F$

$F \rightarrow \text{id}$

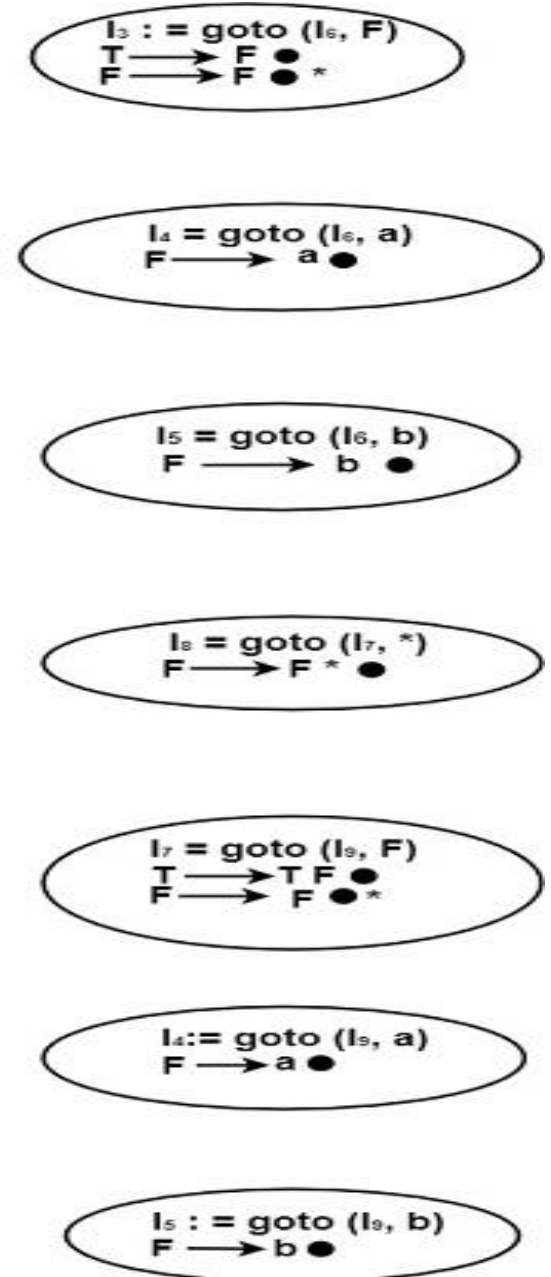
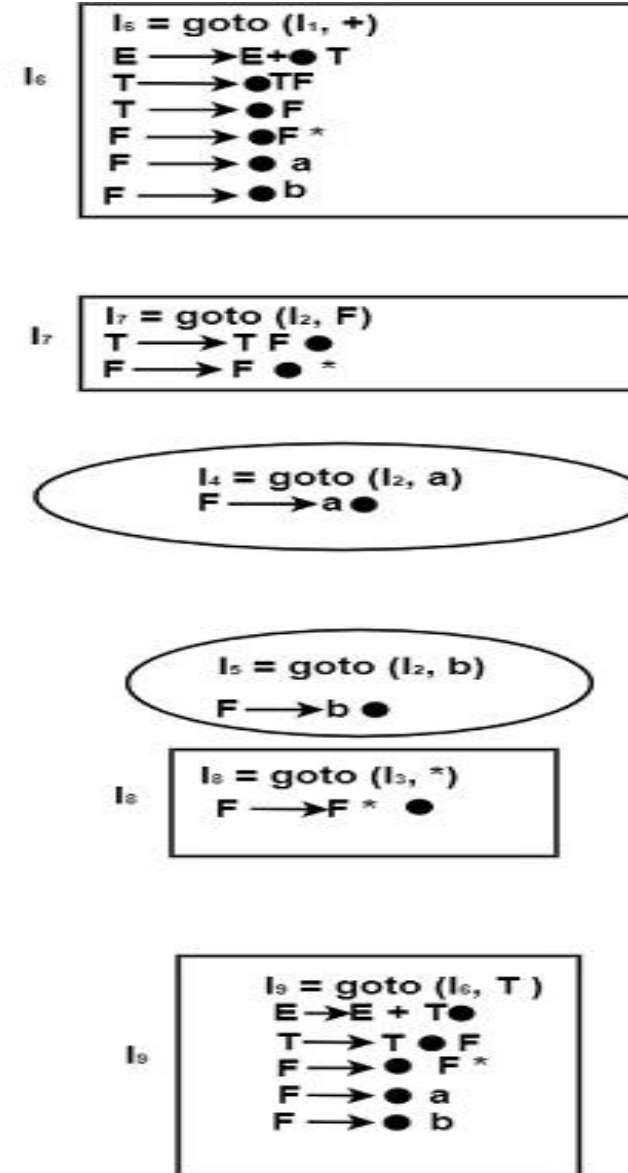
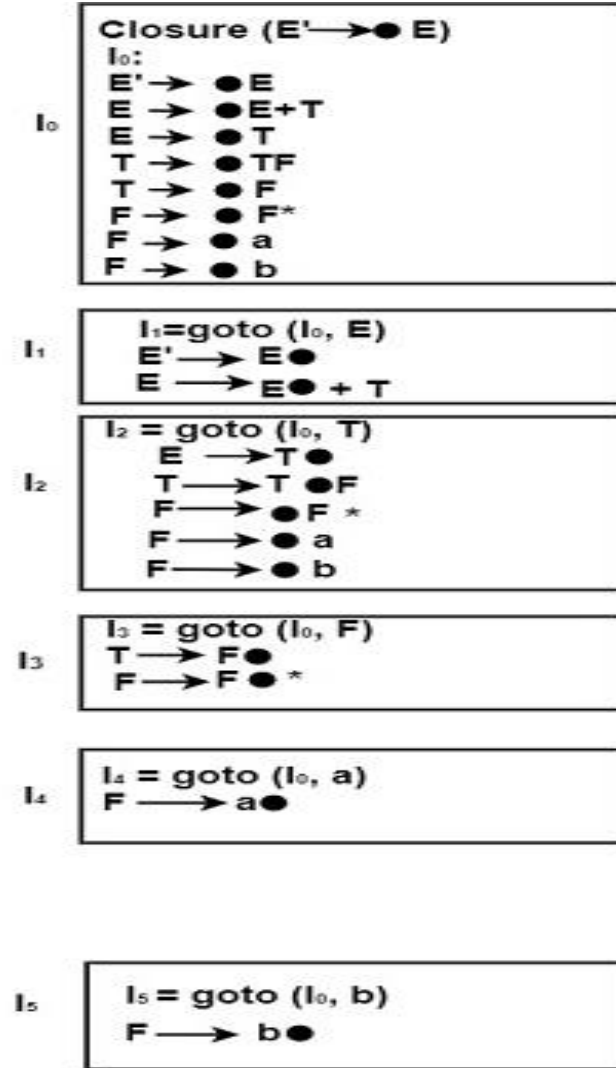


Solution1 – Parsing table

States	Action			Go to			
	id	+	*	\$	E	T	F
I ₀	S ₄				1	2	3
I ₁		S ₅		Accept			
I ₂		R ₂	S ₆	R ₂			
I ₃		R ₄	R ₄	R ₄			
I ₄		R ₅	R ₅	R ₅			
I ₅	S ₄					7	3
I ₆	S ₄						8
I ₇		R ₁	S ₆	R ₁			
I ₈		R ₃	R ₃	R ₃			

Solution2 – Canonical Form

$E \rightarrow E + T | T$
 $T \rightarrow TF | F$
 $F \rightarrow F^* | a | b$



Solution2 – Parsing table

LR Parsing Table

State	Action					goto		
	+	*	a	b	\$	E	T	F
0			s4	s5		1	2	3
1	s6				accept			
2	r2		s4	s5	r2			7
3	r4	s8	r4	r4	r4			
4	r6	r6	r6	r6	r6			
5	r6	r6	r6	r6	r6			
6			s4	s5			9	3
7	r3	s8	r3	r3	r3			
8	r5	r5	r5	r5	r5			
9	r1		s4	s5	r1			7

Solution2 – Parsing the String

Stack	Input String	Action
0	a * b + a \$	Shift
0 a 4	* b + a \$	Reduce by $F \rightarrow a$.
0 F 3	* b + a \$	Shift
0 F 3 * 8	b + a \$	Reduce by $F \rightarrow F *$
0 F 3	b + a \$	Reduce by $T \rightarrow F$
0 T 2	b + a \$	Shift
0 T 2 b 5	+a \$	Reduce by $F \rightarrow b$
0 T 2 F 7	+a \$	Reduce by $T \rightarrow TF$
0 T 2	+a \$	Reduce by $E \rightarrow T$
0 E 1	+a \$	Shift
0 E 1 + 6	a\$	Shift
0 E 1 + 6 a 4	\$	Reduce by $F \rightarrow a$
0 E 1 + 6 F 3	\$	Reduce by $T \rightarrow F$
0 E 1 + 6 T 9	\$	Reduce by $E \rightarrow E + T$
0 E 1	\$	Accept

Solution3 – canonical form

$S \rightarrow L = R \mid R$

$L \rightarrow * R \mid id$

$R \rightarrow L$

Solution:

Step 1: Grammar Augmentation

$S' \rightarrow .S$... rule 0

$S \rightarrow .L = R$... rule 1

$S \rightarrow .R$... rule 2

$L \rightarrow .* R$... rule 3

$L \rightarrow .id$... rule 4

$R \rightarrow .L$... rule 5

Step 2: Closure operation ($S' \rightarrow .S$)

$S' \rightarrow .S$

$S \rightarrow .L = R$

$S \rightarrow .R$

$L \rightarrow .* R$

$L \rightarrow .id$

$R \rightarrow .L$

Goto (i0, S) = i1

$S' \rightarrow S.$

Goto (i0, L) = i2

$S \rightarrow L . = R$

$R \rightarrow L . // **$

Goto (i0, R) = i3

$S \rightarrow R . // **$

Goto (i0, *) = i4

$L \rightarrow * . R$

$R \rightarrow . L$

$L \rightarrow . * R$

$L \rightarrow . id$

Goto (i0, id) = i5

$L \rightarrow id .$

Goto (i2, =) = i6

$S \rightarrow L = . R$

$R \rightarrow . L$

$L \rightarrow . * R$

$L \rightarrow . id$

Goto (i4, R) = i7

$L \rightarrow * R .$

Goto (i4, L) = i8

$R \rightarrow L . // ****$

Goto (i4, *) = i4

Goto (i4, id) = i5

Goto (i6, R) = i9

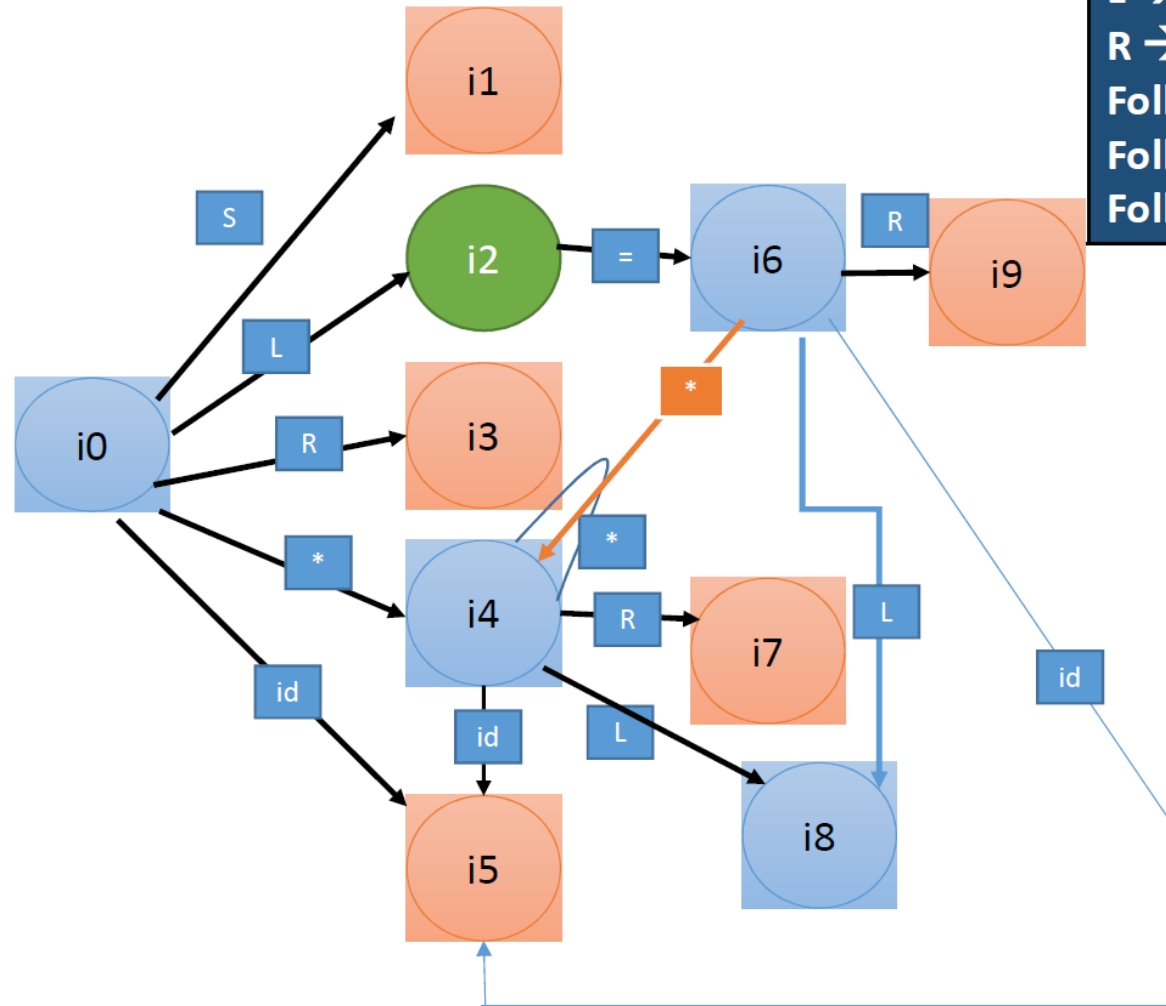
$S \rightarrow L = R .$

Goto (i6, L) = i8

Goto (i6, *) = i4

Goto (i6, id) = i5

Solution3 – cont...



$S \rightarrow L = R \mid R$
 $L \rightarrow * R \mid id$
 $R \rightarrow L$
 $\text{Follow}(S) = \{\$ \}$
 $\text{Follow}(L) = \{=, \$ \}$
 $\text{Follow}(R) = \{=, \$ \}$

Solution3 – parsing table

	=	*	id	\$	S	L	R
I0		S4	S5		1	2	3
I1				Accept			
I2	S6, R5			R5			
I3				R2			
I4		S4	S5			8	7
I5	R4			R4			
I6		S4	S5			8	9
I7	R3			R3			
I8	R5			R5			
I9				R1			

CLR Parser

How to calculate look ahead?

Closure(I)

$S' \rightarrow .S, \$$

$S \rightarrow .CC, \$$

$C \rightarrow .cC, c|d$

$C \rightarrow .d, c|d$

$S \rightarrow CC$

$C \rightarrow cC \mid d$

S'	$\rightarrow$		.	S		,	$\$$
A	$\rightarrow$	α	.	X	β	,	a

Lookahead = First(βa)

First($\$$)

= $\$$

S	$\rightarrow$		.	C	C	,	$\$$
A	$\rightarrow$	α	.	X	β	,	a

Lookahead = First(βa)

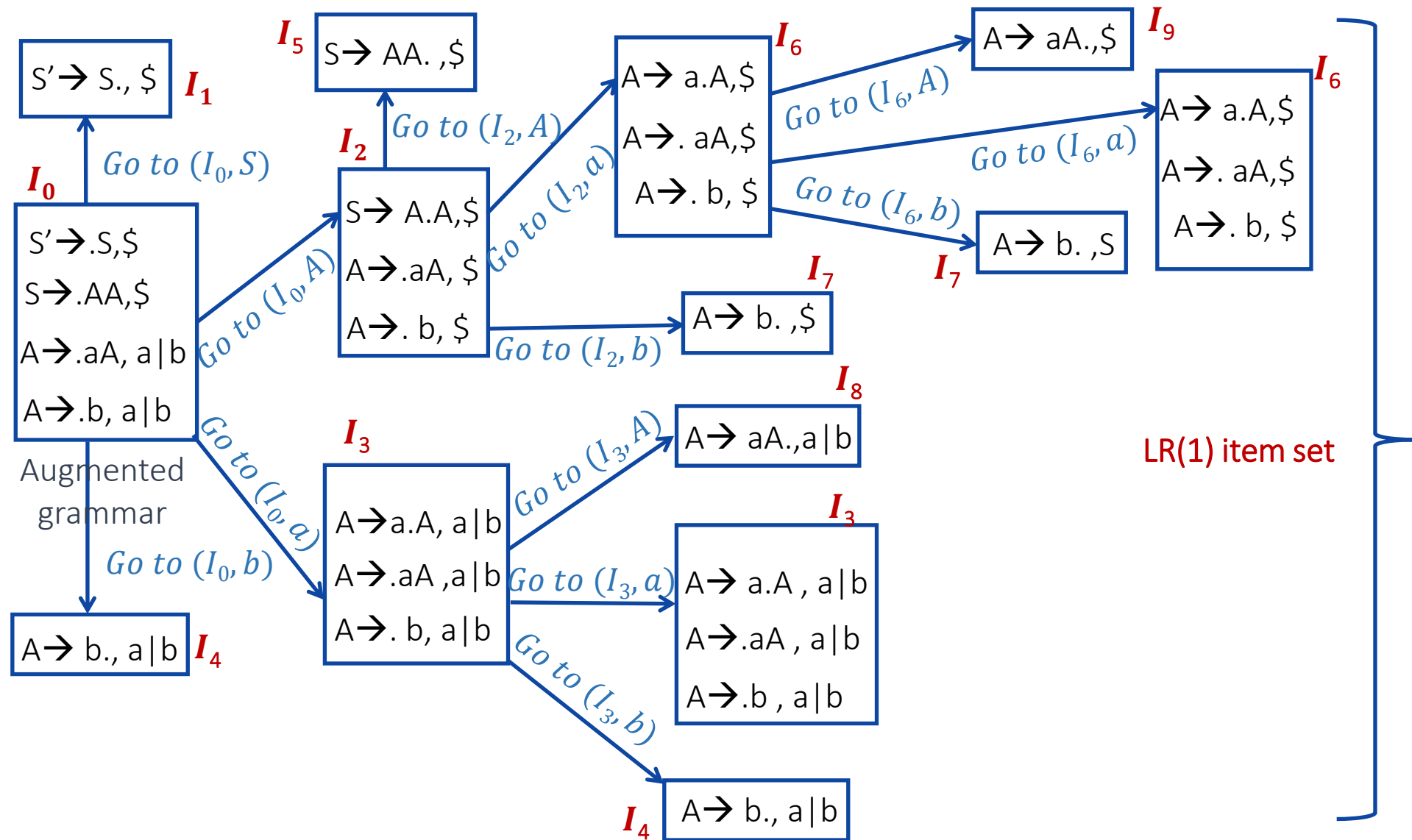
First($C\$$)

= c, d

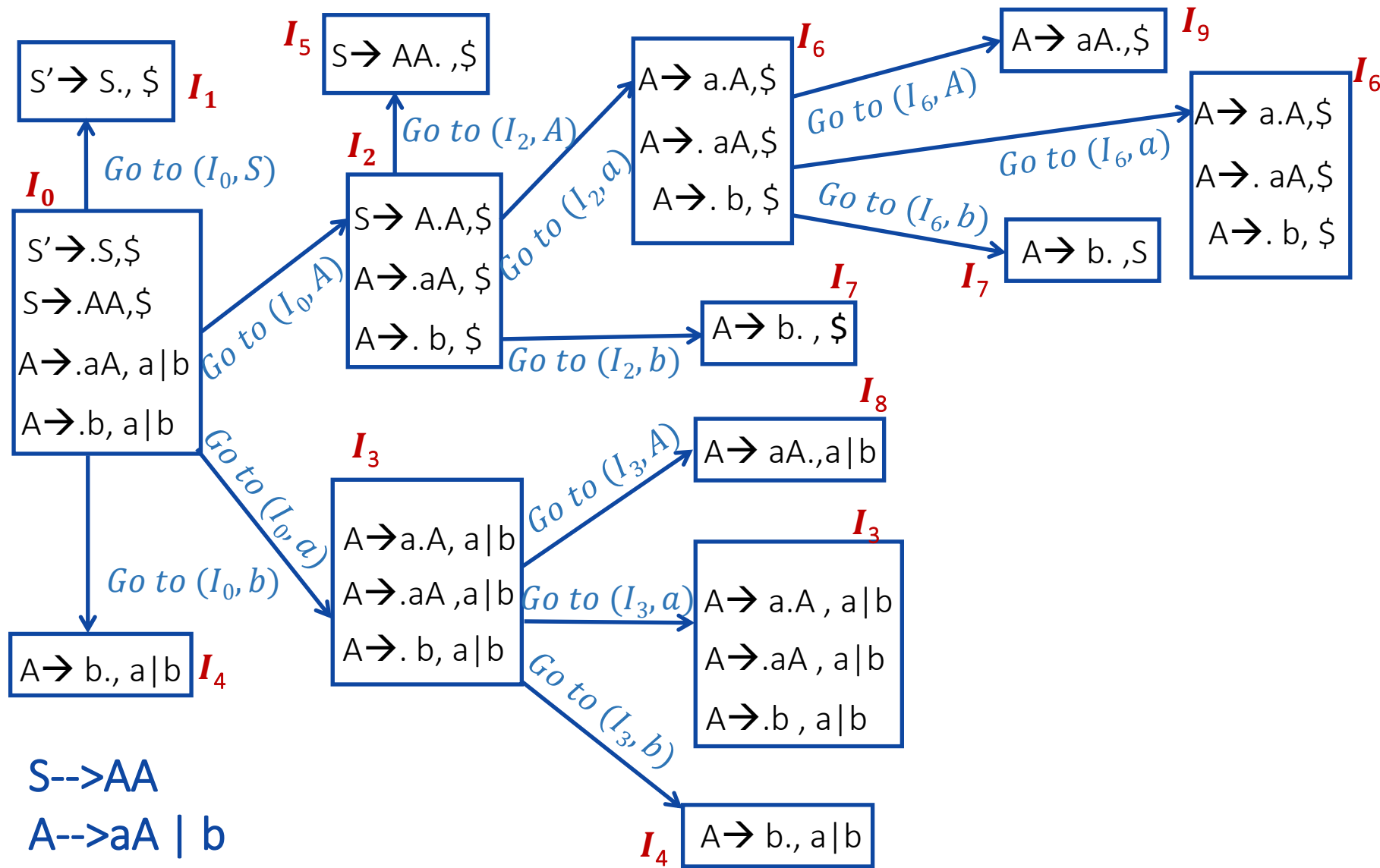
Example: CLR(1)- Canonical LR – LR(0)+LOOKAHEAD

$S \rightarrow AA$

$A \rightarrow aA \mid b$



Example: CLR(1)- canonical LR



Item set	Action			Go to	
	a	b	\$	S	A
0	S3	S4		1	2
1			Accept		
2	S6	S7			5
3	S3	S4			8
4	R3	R3			
5			R1		
6	S6	S7			9
7			R3		
8	R2	R2			
9			R2		

Example 2

Construct CLR parsing table for the following grammar:

$$S \rightarrow CC$$
$$C \rightarrow cC \mid d$$

1) $S \rightarrow CC$
 2) $C \rightarrow cC$
 3) $C \rightarrow d$

Augmented Grammar
 $S' \rightarrow S$
 $S \rightarrow CC$
 $C \rightarrow cC$
 $C \rightarrow d$

LR(1) Items
 $I_0: S' \rightarrow \cdot S, \$$
 $S \rightarrow \cdot CC, \$$
 $C \rightarrow \cdot cC, c|d$
 $C \rightarrow \cdot d, c|d$

goto(I_0, S)
 $I_1: S' \rightarrow S \cdot, \$$

goto(I_0, C)
 $I_2: S \rightarrow \cdot CC, \$$
 $C \rightarrow \cdot cC, \$$
 $C \rightarrow \cdot d, \$$

goto(I_0, c)
 $I_3: C \rightarrow c \cdot C, c|d$
 $C \rightarrow \cdot cC, c|d$
 $C \rightarrow \cdot d, c|d$

goto(I_0, d)
 $I_4: C \rightarrow d \cdot, c|d$
goto(I_2, C)
 $I_5: S \rightarrow CC \cdot, \$$

goto(I_2, c)
 $I_6: C \rightarrow c \cdot C, \$$
 $C \rightarrow \cdot cC, \$$
 $C \rightarrow \cdot d, \$$

goto(I_2, d)
 $I_7: C \rightarrow d \cdot, \$$
goto(I_3, C)
 $I_8: C \rightarrow c C \cdot, c|d$

goto(I_6, C)
 $I_9: C \rightarrow c C \cdot, \$$

State	Action			goto	
	c	d	\$	S	C
0	S3	S4		1	2
1			acc		
2	S6	S7			5
3	S3	S4			8
4	r3	r3			
5			r1		
6	S6	S7			9
7			r3		
8	r2	r2			
9			r2		

Exercise

Construct CLR parsing table for the following grammar:

$S \rightarrow aAd \mid bBd \mid aBe \mid bAe$

$A \rightarrow c$

$B \rightarrow c$

Solution

AUGMENTED GRAMMAR :

$$S' \rightarrow S \quad \text{---(0)}$$

$$S \rightarrow aAd \quad \text{---(1)}$$

$$S \rightarrow bBd \quad \text{---(2)}$$

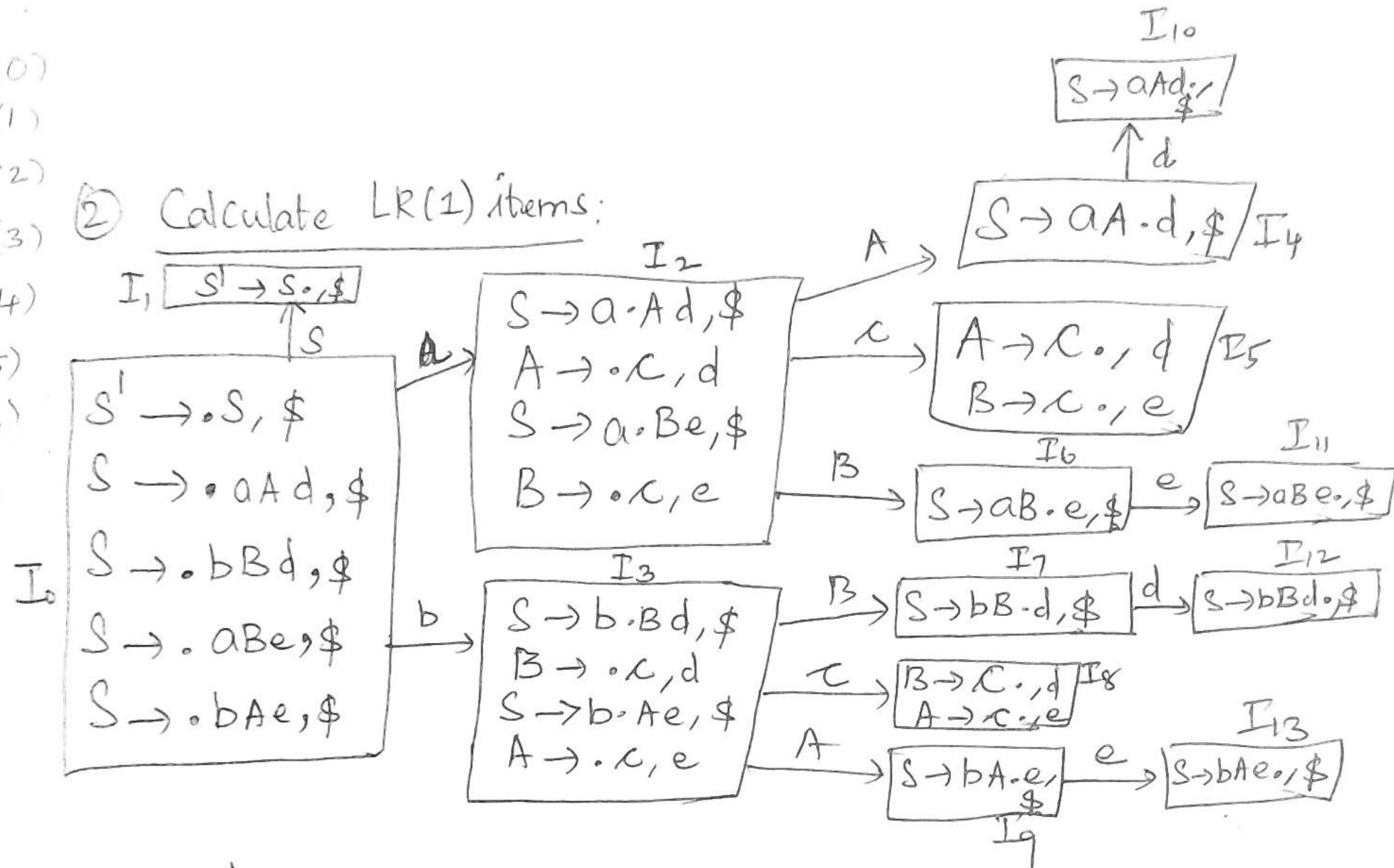
$$S \rightarrow aBe \quad \text{---(3)}$$

$$S \rightarrow bAe \quad \text{---(4)}$$

$$A \rightarrow \epsilon \quad \text{---(5)}$$

$$B \rightarrow \epsilon \quad \text{---(6)}$$

② Calculate LR(1) items:



Cont...

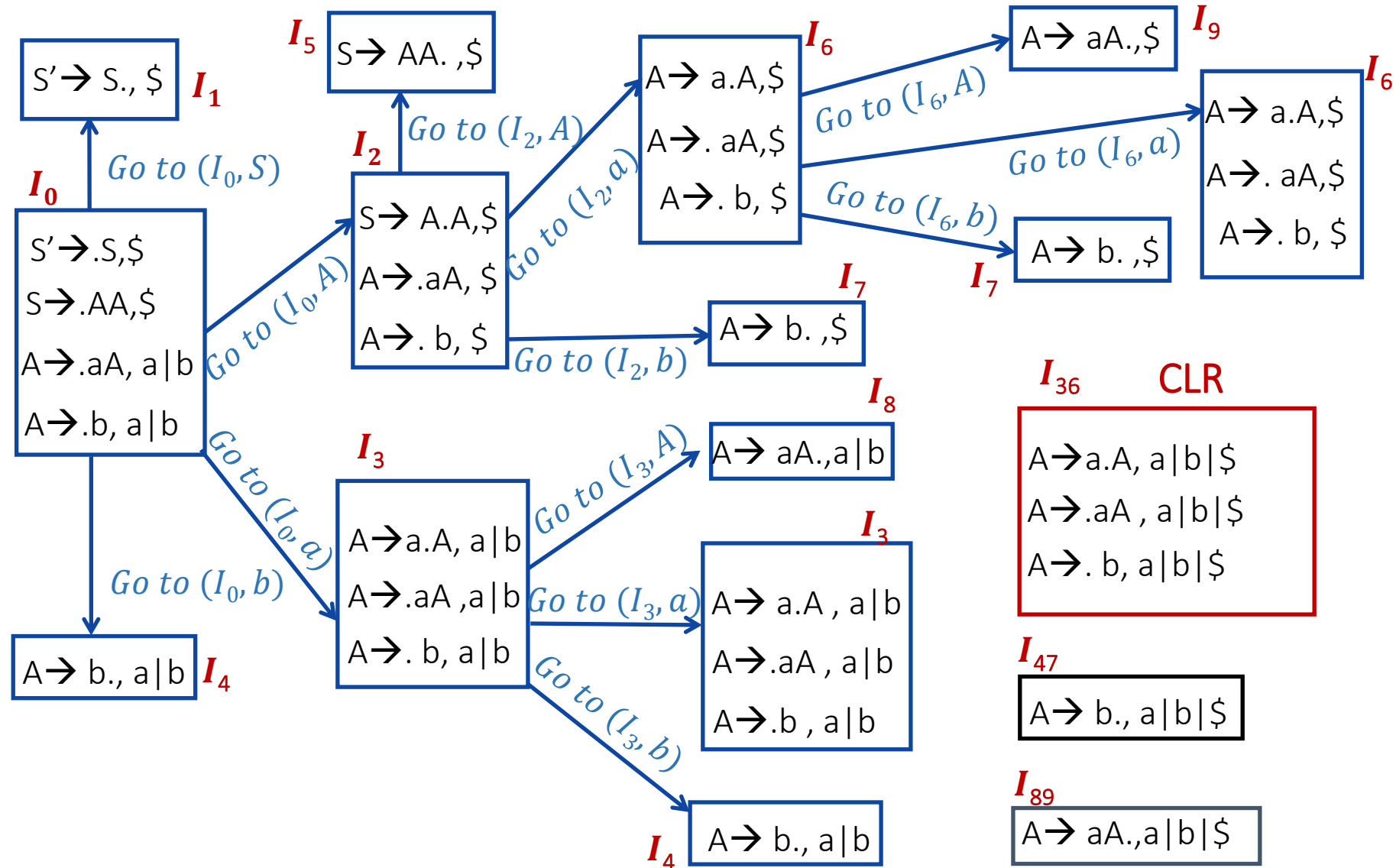
③ Parsing table:

	e	a	b	c	d	\$	S	A	B
0		S2	S3				1		
1						ACCE -PL			
2				S5			4	6	
3				S8			9	7	
4					S10				
5	R5 RL				R5 RL				
6	S11								
7					S12				
8	R5 RL				R5 RL				
9	S12								
10						R1			
11						R3			
12						R2			
13						R4			

LALR Parser

- LALR Parser is lookahead LR parser.
- It is the most powerful parser which can handle large classes of grammar.
- The size of CLR parsing table is quite large as compared to other parsing table.
 - LALR reduces the size of this table.
- LALR works similar to CLR. The only difference is, it combines the similar states of CLR parsing table into one single state.

Example: LALR(1)- look ahead LR



$S \rightarrow AA$
 $A \rightarrow aA \mid b$

Example: LALR(1)- look ahead LR

Item set	Action			Go to	
	a	b	\$	S	A
0	S3	S4		1	2
1			Accept		
2	S6	S7			5
3	S3	S4			8
4	R3	R3			
5			R1		
6	S6	S7			9
7			R3		
8	R2	R2			
9			R2		

CLR Parsing Table



Item set	Action			Go to	
	a	b	\$	S	A
0	S36	S47		1	2
1			Accept		
2	S36	S47			5
36	S36	S47			89
47	R3	R3	R3		
5			R1		
89	R2	R2	R2		

LALR Parsing Table

Construct LALR (1) Parsing table for the grammar.

$S \rightarrow A a \mid b A c \mid d c \mid b d a$

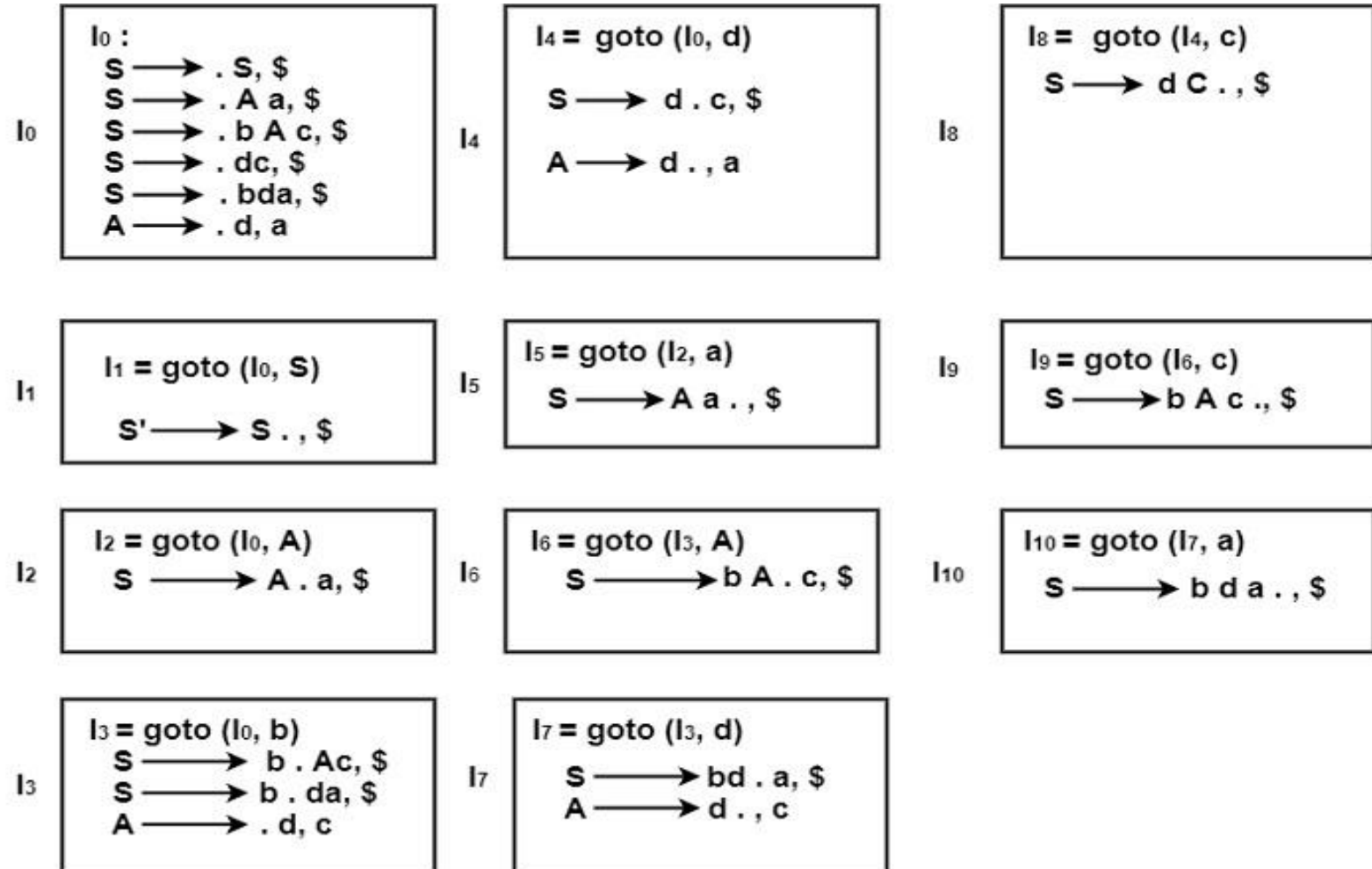
$A \rightarrow d$

Parse input string "bdc" using LALR Parsing Table.

Step1- Construct Augmented Grammar

- (0) $S' \rightarrow S$
- (1) $S \rightarrow A a$
- (2) $S \rightarrow b A c$
- (3) $S \rightarrow d c$
- (4) $S \rightarrow b d a$
- (5) $A \rightarrow d$

Step2-Find the canonical set of LR (1) items for the Grammar



Cont...

Step3 – LALR Parsing Table

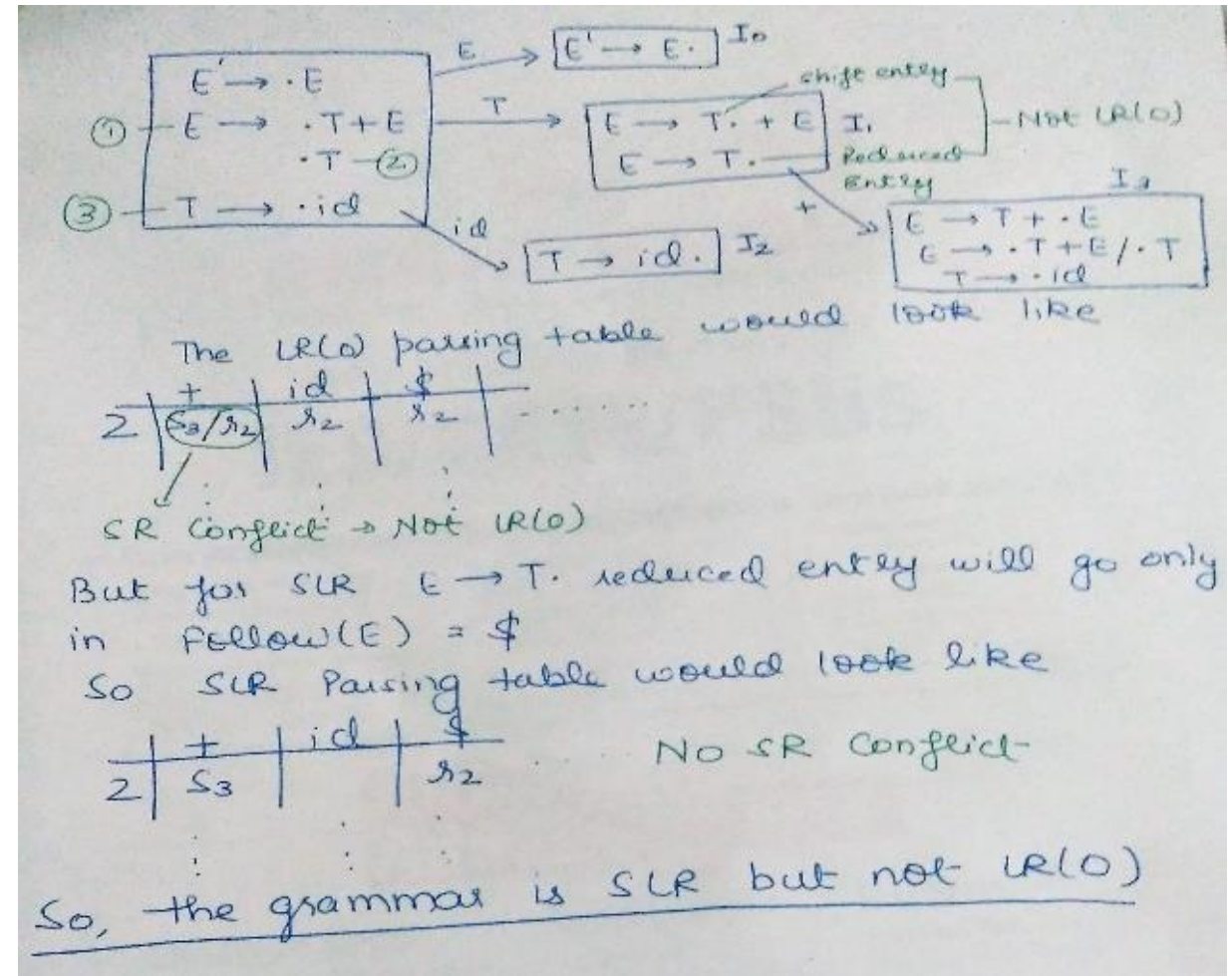
States	Action						goto
	a	b	c	d	\$	S	A
0		s3		s4		1	2
1					accept		
2	s5						
3				s7			6
4	r5		s8				
5					r1		
6			s9				
7	s10		r5				
8					r3		
9					r2		
10					r4		

Step4 – Parsing of String "bdc"

Stack	Input String	Action
\$ 0	bdc \$	Shift 3
\$ 0 b 3	dc \$	Shift 7
\$ 0 b 3 d 7	c \$	Reduce by $A \rightarrow d$
\$ 0 b 3 A 6	c \$	Shift 9
\$ 0 b 3 A 6 c 9	\$	Reduce by $S \rightarrow b A c$
\$ 0 s 1	\$	accept

Points to be noted – not SLR

- In case of **two reductions**, if the follow of both the reduced productions have something common then it will result in **multiple entries** in table hence **not SLR**.
- In case of **one shift and one reduction**, if there is a GOTO operation from that state on a terminal which is the follow of the reduced production then it will result in **multiple entries** hence **not SLR**.



How do you identify whether you can construct LL(1), LR(0), or SLR(1)?

$X \rightarrow Yz \mid a$

$Y \rightarrow bZ \mid \varepsilon$

$Z \rightarrow \varepsilon$

To check if you can construct LR(0) or SLR(1), we begin by building up all of the LR(0) configuring sets for the grammar.

```
(1)
X' -> .X
X -> .Yz [$]
X -> .a [$]
Y -> . [z]
Y -> .bZ [z]
```

```
(1)
X' -> .X
X -> .Yz
X -> .a
Y -> .
Y -> .bZ
```

```
(2)
X' -> X.
```

```
(3)
X -> Y.z
```

```
(4)
X -> Yz.
```

```
(5)
X -> a.
```

```
(6)
Y -> b.Z
Z -> .
```

```
(7)
Y -> bZ.
```


Points to be noted - CLR

Consider the following grammar

$S \rightarrow AaAb \mid BbBa$

$A \rightarrow \epsilon$

$B \rightarrow \epsilon$

Augmented grammar –

$S' \rightarrow S$

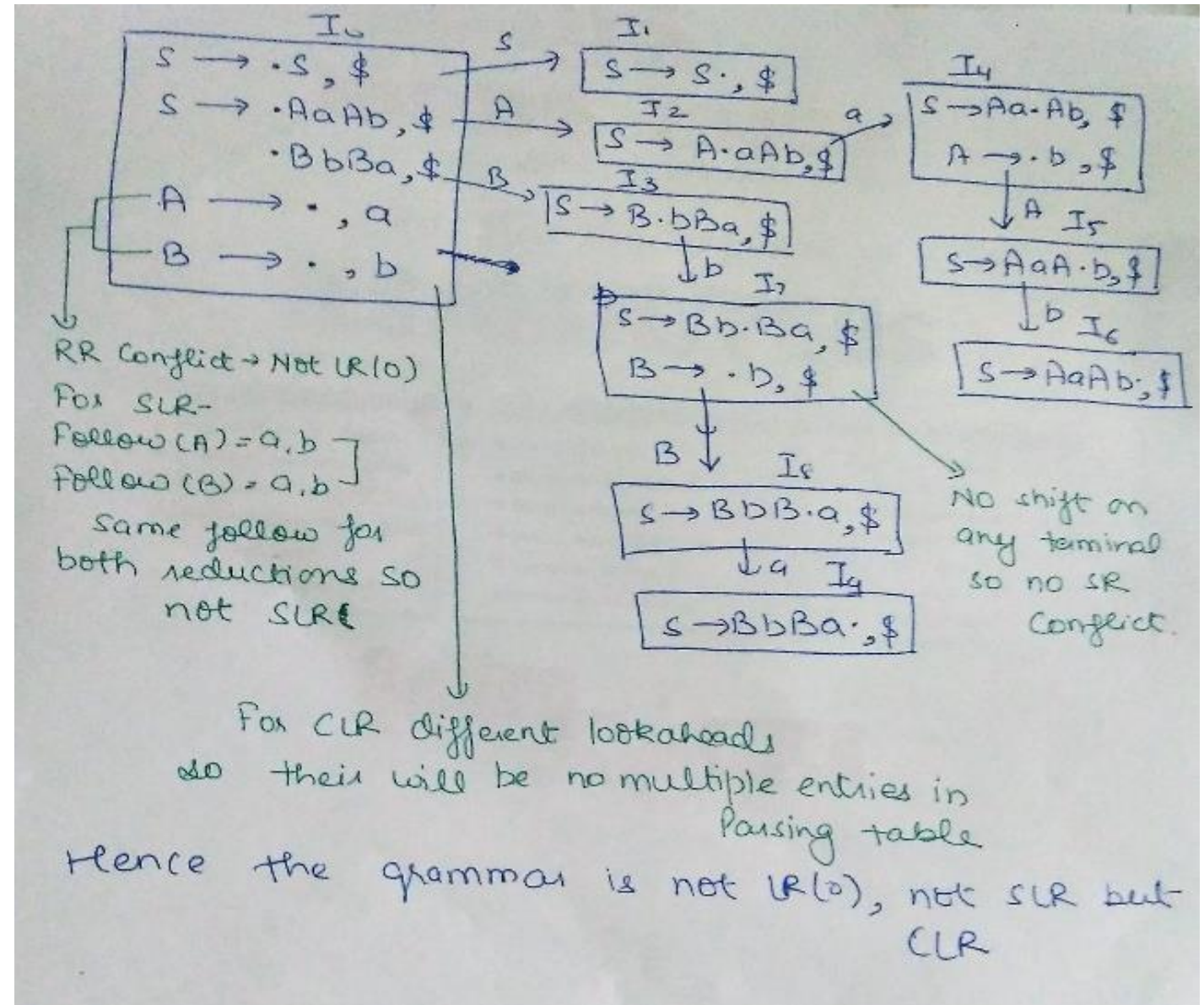
$S \rightarrow AaAb$

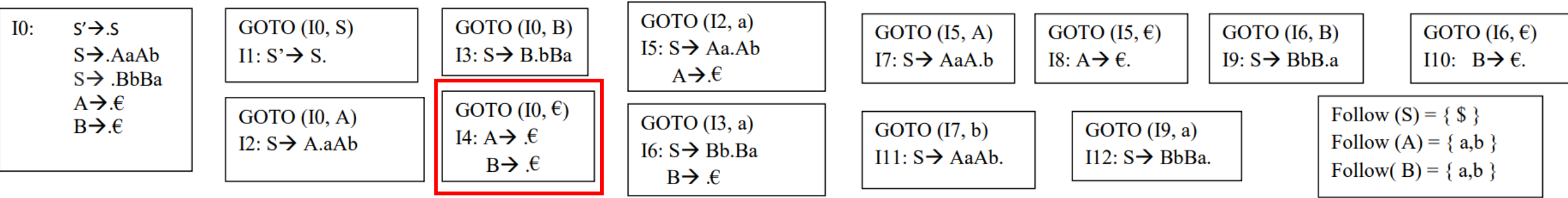
$S \rightarrow BbBa$

$A \rightarrow \epsilon$

$B \rightarrow \epsilon$

GOTO graph for this grammar will be -





Step 3: Construction of an SLR parse table

State	ACTION				GOTO		
	a	b	ϵ	\$	S	A	B
0			S4		1	2	3
1				Accept			
2	S5						
3		S6					
4	r3/r4	r3/r4					
5			S8			7	
6			S10				9
7		S11					
8	r3	r3					
9	S12						
10	r4	r4					
11				r1			
12				r1			

Cont...

- If a state has two reductions and both have same lookahead then it will in multiple entries in parsing table thus a conflict.
- If a state has one reduction and there is a shift from that state on a terminal same as the lookahead of the reduction then it will lead to multiple entries in parsing table thus a conflict.

1. Even though CLR parser does not have RR conflict but LALR may contain RR conflict.
2. If number of states $LR(0) = n1$,
number of states $SLR = n2$,
number of states $LALR = n3$,
number of states $CLR = n4$ then,
 $n1 = n2 = n3 \leq n4$

